

Proyecto: API REST de Gestión de Biblioteca

Descripción del Proyecto

La aplicación permitirá gestionar una biblioteca que incluye libros, autores, y préstamos de libros a usuarios. Tendrá las siguientes funcionalidades:

1. Gestión de Libros:

- Crear, leer, actualizar y eliminar libros.
- Filtrar por título, género, autor o año de publicación.
- Implementar paginación y ordenación de resultados.

2. Gestión de Autores:

- Crear, leer, actualizar y eliminar autores.
- Relación uno-a-muchos entre autor y libros (un autor puede escribir varios libros).

3. Gestión de Usuarios y Préstamos:

- Registrar usuarios que puedan tomar libros en préstamo.
- Funcionalidad para que los usuarios pidan un libro prestado.
- Registrar cuándo se realizó un préstamo, y controlar el estado del libro (prestado o disponible).

4. Autenticación y Autorización:

- Autenticación de usuarios para que puedan tomar libros en préstamo (JWT).
- Autenticación de bibliotecarios que pueden gestionar libros y autores.
- Control de acceso basado en roles: roles de **Usuario** y **Bibliotecario**.

Funcionalidades Clave a Implementar

1. Entidades (JPA):

- **Libro:** 'id', 'titulo', 'autor', 'genero', 'añoPublicacion', 'estado' (disponible o prestado).
- **Autor:** 'id', 'nombre', 'nacionalidad', 'libros' (relación uno-a-muchos con libros).
- **Usuario:** 'id', 'nombre', 'email', 'password', 'rol' (usuario o bibliotecario).
- **Préstamo:** 'id', 'libroId', 'usuarioId', 'fechaPrestamo', 'fechaDevolucion'.

2. Endpoints REST:

- **Libros:**
 - 'GET /libros': listar todos los libros (con paginación y filtros).
 - 'GET /libros/{id}': obtener detalles de un libro.
 - 'POST /libros': crear un nuevo libro (solo bibliotecarios).

- ‘PUT /libros/{id}’: actualizar un libro existente (solo bibliotecarios).
- ‘DELETE /libros/{id}’: eliminar un libro (solo bibliotecarios).
- **Autores:**
 - ‘GET /autores’: listar todos los autores.
 - ‘POST /autores’: crear un nuevo autor (solo bibliotecarios).
 - ‘PUT /autores/{id}’: actualizar un autor existente (solo bibliotecarios).
 - ‘DELETE /autores/{id}’: eliminar un autor (solo bibliotecarios).
- **Usuarios:**
 - ‘POST /usuarios’: registrar un nuevo usuario.
 - ‘GET /usuarios/me’: obtener la información del usuario autenticado.
- **Préstamos:**
 - ‘POST /prestamos’: solicitar un préstamo de libro (solo usuarios).
 - ‘GET /prestamos’: listar los préstamos del usuario autenticado (solo usuarios).
 - ‘PUT /prestamos/{id}/devolver’: devolver un libro (solo usuarios).

3. Páginas y filtros:

- **Libros:** Implementar paginación y filtros para libros (por autor, título, género).
 - Ejemplo de uso: ‘GET /libros?titulo=Spring&autor=John&page=0&size=10&sort=titulo,asc’.

4. Autenticación con JWT:

- Implementar un sistema de autenticación basado en JWT donde los usuarios obtienen un token tras iniciar sesión.
- Los bibliotecarios tienen acceso a los endpoints de gestión (crear, actualizar, eliminar), mientras que los usuarios solo pueden acceder a los endpoints de préstamo.

5. Pruebas Unitarias e Integración:

- Escribir pruebas para verificar la funcionalidad básica de los CRUDs.
- Pruebas para las restricciones de acceso según roles (usuario o bibliotecario).
- Pruebas para asegurarse de que los préstamos solo se permiten para libros disponibles.

Pasos para Implementar la API REST

1. Configuración Inicial

- Crear un proyecto **Spring Boot** con dependencias necesarias:
 - ‘spring-boot-starter-web’: Para crear la API REST.
 - ‘spring-boot-starter-data-jpa’: Para la persistencia de datos con JPA.

- 'spring-boot-starter-security': Para implementar la seguridad.
- 'spring-boot-starter-validation': Para validar las entradas del usuario.
- 'jjwt': Para gestionar la autenticación con JWT.
- **Base de Datos:** Puedes usar **H2** en memoria durante el desarrollo o **MySQL** para un proyecto más robusto.

2. Modelo de Datos (Entidades JPA)

- Crear las entidades 'Libro', 'Autor', 'Usuario' y 'Préstamo'.
- Definir las relaciones entre entidades, como **uno-a-muchos** entre **Autor** y **Libro**.

```
@Entity
public class Libro {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;
    private String genero;
    private String estado; // disponible o prestado

    @ManyToOne
    private Autor autor;

    // getters y setters
}
```

3. Controladores REST

- Crear los controladores para cada entidad: **LibroController**, **AutorController**, **UsuarioController**, y **PrestamoController**.
- Usar anotaciones como '@GetMapping', '@PostMapping', '@PutMapping', y '@DeleteMapping' para definir los endpoints.
- Ejemplo de un controlador básico para libros:

```
@RestController
@RequestMapping("/libros")
public class LibroController {

    @Autowired
    private LibroService libroService;

    @GetMapping
    public Page<Libro> listarLibros(@RequestParam Optional<String> titulo, Pageab
```

```

        return libroService.listarLibros(titulo, pageable);
    }

    @PostMapping
    @PreAuthorize("hasRole('BIBLIOTECARIO')")
    public ResponseEntity<Libro> crearLibro(@RequestBody @Valid Libro libro) {
        return ResponseEntity.ok(libroService.guardar(libro));
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('BIBLIOTECARIO')")
    public ResponseEntity<Libro> actualizarLibro(@PathVariable Long id, @RequestBody Libro libro) {
        return ResponseEntity.ok(libroService.actualizar(id, libro));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('BIBLIOTECARIO')")
    public ResponseEntity<Void> eliminarLibro(@PathVariable Long id) {
        libroService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}

```

4. Seguridad con JWT

- Configurar Spring Security para autenticar usuarios y proteger los endpoints de bibliotecarios.
- Implementar JWT para manejar la autenticación, con endpoints para **login** y **registro de usuarios**.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    // Configuración de JWT y reglas de autorización
}

```

5. Pruebas Unitarias e Integración

- Implementar pruebas unitarias con **JUnit** y **Mockito** para comprobar el comportamiento de los controladores y servicios.
- Escribir pruebas de integración para asegurar que los endpoints funcionan correctamente.

6. Documentación con Swagger

- Integrar **Swagger** para generar automáticamente la documentación de la API.

- Añadir la configuración en el proyecto para que los usuarios puedan ver y probar los endpoints desde el navegador.

```
@Configuration
@EnableSwagger2
public class SwaggerConfig {
    // Configuración de Swagger
}
```

Evaluación del Proyecto

- Los estudiantes deberán implementar las funcionalidades mencionadas, incluyendo autenticación, manejo de roles, pruebas, y documentación.
- Se evaluará la calidad del código, la eficiencia de la API, la correcta implementación de seguridad, y la capacidad de gestionar datos con paginación y filtros.

Este proyecto cubre todos los aspectos importantes que los estudiantes necesitarán aprender en las clases, desde la creación básica de una API REST hasta la seguridad y el despliegue. Además, les proporcionará una experiencia completa en el desarrollo de una aplicación back-end real.